Computer Networks - Assignment 2

TCP based web Application

Overview:

This Python project implements a basic TCP based web application. The server initially handles a single client connection at a time in part_1.py. In part_2.py, the server is enhanced to handle multiple concurrent connections using multithreading, significantly improving its performance and responsiveness.

Both server components efficiently process incoming HTTP GET requests, retrieve the requested files from the local filesystem, and send the appropriate HTTP response, including status codes and content. The server supports a special filename "BreakOut" to trigger a graceful shutdown and includes basic error handling mechanisms for robust operation.

Initially, the server was designed to be accessed directly from web browsers by entering the appropriate URL in the address bar. Later, a dedicated client program client.py was created to provide more flexibility and control over the interaction with the server. The client program allows users to specify the server's hostname, port, and the desired filename to request. It establishes a connection to the server, sends the

HTTP GET request, and receives the server's response, providing a more programmatic way to interact with the file server.

Code Components:

Part_1.py(Server for part 1):

- **Initialization:** The server imports necessary modules, defines constants, and creates a TCP socket.
- **Binding and Listening:** The server binds the socket to a specific IP and port and starts listening for incoming connections.
- **Main Loop:**
    - The server enters an infinite loop to continuously accept new client connections.
    - For each incoming connection:
        - The server receives data from the client.
        - The received message is parsed to extract the requested filename.
        - If the filename is "BreakOut", the server sends a 410 Gone(this is not actually the real implementation of these codes by http, but we used it to our convenience here) response and closes the connection.
        - If the file exists, the server sends a 200 OK response and the file content.
        - If the file doesn't exist, the server sends a 404 Not Found response.
        - The server handles exceptions and sends appropriate error responses.
        - The client connection is closed.
- **Explanation:**

- This server handles one client connection at a time.
- It listens for incoming connections and processes requests sequentially.
- The server can serve requested files or send error responses based on the filename.
- The "BreakOut" filename is used as a signal to terminate the server.

part_2.py(Server for part 2):

- **Initialization:** The server imports necessary modules, defines constants, and creates a TCP socket.
- **Binding and Listening:** The server binds the socket to a specific IP and port and starts listening for incoming connections.
- **Main Loop:**
  - The server enters an infinite loop to continuously accept new client connections.
  - For each incoming connection:
    - A new thread is created to handle the client connection.
    - The thread's target function is `handle_client`.
- **`Handle_client` function:**
  - Receives data from the client.
  - Parses the message to extract the requested filename.
  - If the filename is "BreakOut":
    - Sends a 410 Gone response(this is not actually the real implementation of these codes by http, but we used it to our convenience here) and closes the connection.
  - If the file exists:
    - Sends a 200 OK response and the file content.
  - If the file doesn't exist:
    - Sends a 404 Not Found response.
  - Handles exceptions and sends appropriate error responses.
  - Closes the client connection.

- **Explanation:**
    - This server is multi-threaded, allowing it to handle multiple client connections concurrently.
    - Each incoming connection is assigned to a separate thread, improving performance and responsiveness.
    - The `handle_client` function handles the actual processing of requests within each thread.
    - The overall flow is similar to `part_1.py`, but with the added concurrency.

client.py(Client):

- **Initialization:** The client imports necessary modules, parses command-line arguments, and creates a TCP socket.
- **Connection Establishment:** The client connects to the specified server over the specified port.
- **Request Sending:** The client connects to the specified server over the specified port.
- **Response Reception:** The client receives the server's response by first initializing the handshaking with the server.
- **Response Parsing:** The client parses the received response to extract the status code and content.
- **Output:** The client prints the received status code and content to the console.
- **Connection Closing:** The client closes the socket.
- **Explanation:**
    - The client program serves as an interface for interacting with the server.
    - It takes command-line arguments to specify the server details and the requested filename.
    - The client establishes a TCP connection to the server.
    - It sends an HTTP GET request, including the filename and client information.

    - The client receives the server's response, which includes the status code and content.
    - The client parses the response to extract the relevant information.
    - The client prints the results to the console, providing feedback to the user.
    - Finally, the client closes the socket to terminate the connection.

Assumptions and Limitations:

- **Server Availability:** The client assumes that the server is running and accessible at the specified address and port.
- **TCP Connection:** The client assumes a TCP-based connection with the server, ensuring reliable data delivery and flow control.
- **HTTP Protocol Understanding:** The client is expected to understand the HTTP protocol and correctly interpret the server's responses.
- **Network Connectivity:** The client assumes a reliable network connection between the client and server.
- **Error Handling:** The client and server may have limited error handling capabilities, and unexpected responses or errors might not be handled gracefully.